

Proyecto Tienda Vecina

Sumativa 1: Informe de planificación del proyecto

NOMBRES: Misael Ancán

Christian Díaz

Juan Sanhueza

Roberto Yáñez

CARRERA: Ingeniería en Ciberseguridad

ASIGNATURA: Desarrollo Seguro (CI3V64)

SECCIÓN: 2026/P CI3V64/V-IEC-N6-P2-C1/V

SEDE: Santiago Sur

PROFESOR: Mario Yáñez Urrutia

FECHA: 10/11/2026



Índice

Contenido

Introducción 3
1. Portada y equipo 3
2. Descripción del proyecto 4
2.1 Problema 4
2.2 Usuarios 4
2.3 Alcance y exclusiones 4
2.4 Tecnología elegida 5
3. Proceso DevOps y DevSecOps del equipo 5
4. Requerimientos 6
4.1 Requerimientos funcionales (historias de usuario) 6
4.2 Requerimientos no funcionales 7
4.3 Requerimientos de seguridad 7
5. Amenazas y diseño 8
5.1 Método 8
5.2 Arquitectura, datos y límites de confianza 9
5.3 Amenazas STRIDE del sistema 9
5.4 Trazabilidad amenaza, requerimiento, historia y prueba 10
6. Planificación en GitHub Projects 11
6.1 Configuración 11
6.2 Estructura del backlog 11
6.3 Sprints, objetivos y responsables 12
6.4 Evidencia en GitHub Projects 12
7. Repositorio en GitHub 13
7.1 Estructura y ramas 13
7.2 Pull requests del periodo 13
7.3 README, .gitignore y secretos 14
7.4 Reglas planificadas 14
7.5 Evidencia en GitHub 14
8. Cronograma y riesgos 15
8.1 Hitos de la primera etapa 15
8.2 Riesgos de esta etapa 15
9. Conclusiones y uso de IA 16
9.1 Aprendizajes 16
9.2 Dificultades 16
9.3 Registro de uso de IA 16
Bibliografía 16



Introducción

Tienda Vecina es un marketplace donde varios vendedores pueden publicar sus productos y los compradores pagan con Webpay Plus de Transbank. La primera versión del sistema es un mockup funcional que corre en local: backend en Django con Django REST Framework, frontend en Vue 3 y base de datos PostgreSQL.

A partir de este mockup se van a completar las funciones que faltan, como la aprobación de vendedores y el historial de órdenes, y se va a llevar la plataforma a un nivel de seguridad que se pueda demostrar con requerimientos, un modelo de amenazas, controles en el código, pruebas y evidencia en el repositorio. En la revisión inicial del mockup se encontraron problemas concretos, por ejemplo que una orden puede quedar pagada aunque Transbank no la autorice, por lo que los requerimientos y las amenazas parten de lo que ya existe.

Este informe presenta la planificación de la primera etapa (semanas 1 a 3), que se gestiona en GitHub Projects con autorización del Product Owner. Lo que todavía no existe o falta capturar aparece marcado en azul como pendiente.

1. Portada y equipo

El equipo tiene cuatro integrantes y los roles se definieron de acuerdo con lo que pidió el Product Owner. Cada rol tiene un responsable principal, pero todos programan, revisan pull requests (PR) y tienen que poder explicar cualquier parte de este informe.

Tabla 1. Integrantes, roles y responsabilidades.

Integrante	Rol	Responsabilidades
Mario Yáñez Urrutia	Product Owner	Prioriza el backlog, resuelve dudas de alcance y acepta las historias al cierre de cada sprint.
Roberto Yáñez	Desarrollador / Analista	Redacta las historias de usuario y los requerimientos no funcionales, dibuja el diagrama de arquitectura y desarrolla las funciones del producto. Dueño del repositorio y autor de los PR #1, #2 y #4.
Juan Sanhueza	Security Champion	Hace la revisión de seguridad del código, mantiene los requerimientos de seguridad y el modelo STRIDE, limpia los secretos de las plantillas y revisa los PR que tocan apps/accounts o apps/payments.
Misael Ancán	DevOps / QA	Configura GitHub Projects, las ramas y su protección, el entorno local y la convención de PR; revisa que cada cambio cumpla los criterios de aceptación antes de pasar a QA y a main. Autor de los PR #3 y #68.
Christian Díaz	Scrum Master / Líder	Dirige la planificación, la revisión y la retrospectiva de cada sprint, hace el seguimiento del tablero, coordina al equipo y con el Product Owner, consolida el informe y lo presenta al Product Owner. También mantiene el README.

Tabla 2. Enlaces del proyecto.

Recurso	Enlace
Repositorio en GitHub	https://github.com/RobertxDessinacap/Tienda_Vecina_INACAP
Tablero en GitHub Projects (reemplaza a Azure Boards, acordado con el Product Owner)	Tienda Vecina - Planificación DevSecOps: https://github.com/users/RobertxDessinacap/projects/2
Acceso del Product Owner	El repositorio es público, así que se puede ver sin invitación. El Project se comparte con el Product Owner.
Ubicación de este informe	Se sube a docs/S1_informe_planificacion.pdf dentro del repositorio

Forma de trabajo: sprints de una semana, con planificación al inicio y revisión al cierre de cada sprint, que dirige el Scrum Master y quedan registradas en GitHub Projects. Para esta etapa son tres sprints (semanas 1 a 3). La coordinación se hace por Discord.



2. Descripción del proyecto

2.1 Problema

Un vendedor pequeño que quiere vender por internet en Chile tiene dos caminos: publicar en un marketplace grande, con comisiones altas, o armar una tienda propia sin saber bien qué tan segura es. Tienda Vecina apunta a ese segundo caso: un marketplace simple, con catálogo por categorías, carrito y pago con Webpay Plus, que es el medio de pago más usado en el país.

En las primeras semanas el equipo ya agregó funciones nuevas, integradas a develop por los PR #1 a #4: modal para convertirse en vendedor y filtro por rango de precio; panel del vendedor para editar y dar de baja publicaciones; registro de tienda oficial; reseñas de 1 a 5 estrellas; galería de hasta 5 fotos por producto; un arreglo para que un token vencido no bloquee el catálogo, junto con extender la vida del token a 7 días. Estas funciones se consideran en el alcance y en el modelo de amenazas.

El problema del proyecto tiene dos partes. La primera es completar las funciones que faltan, como la aprobación de vendedores y el historial de órdenes. La segunda, y la más importante, es llevar la plataforma a un nivel de seguridad que se pueda demostrar con pruebas, análisis automáticos y evidencia en el repositorio, y no solo con lo que dice la documentación.

2.2 Usuarios

Tabla 3. Actores del sistema.

Actor	Qué hace en el sistema	Datos que maneja
Comprador anónimo	Navega el catálogo, busca y arma su carrito.	Ninguno personal; el carrito queda en el navegador.
Comprador registrado	Compra, paga con Webpay Plus, consulta sus órdenes y deja reseñas.	Correo, contraseña, nombre, RUT y dirección (opcionales).
Vendedor	Publica, edita y da de baja sus productos; puede registrar una tienda oficial.	Alias público, productos, hasta 5 fotos por producto, nombre y URL de tienda.
Administrador	Aprueba vendedores y administra el catálogo desde /admin.	Acceso a todos los datos del sistema.
Transbank (sistema externo)	Autoriza o rechaza el pago con tarjeta.	Recibe orden y monto; devuelve código de autorización y últimos 4 dígitos.

2.3 Alcance y exclusiones

Dentro del alcance

- Registro e inicio de sesión con JWT y tres roles (comprador, vendedor y administrador), con aprobación de vendedores.
- Catálogo con 14 categorías, búsqueda, filtro por precio y publicación de productos con hasta 5 fotos y descripción ADS de más de 250 caracteres.
- Panel del vendedor (editar y dar de baja publicaciones), tienda oficial y reseñas de compradores.
- Carrito con total calculado en el servidor y pago con Webpay Plus en el ambiente de integración, con comprobante e historial de órdenes.
- Para esta etapa: requerimientos, modelo de amenazas, tablero en GitHub Projects y repositorio configurado. Los controles se implementan en las etapas siguientes.

Fuera del alcance

- Pagos reales: solo se usa el ambiente de integración de Transbank y tarjetas de prueba.
- Despacho, boleta o factura electrónica del SII, devoluciones y reembolsos.
- App móvil nativa. En una etapa posterior la SPA se considera como el cliente que la usaría.
- Certificación ISO/IEC 27001. Se alinean prácticas con sus controles, pero no se declara cumplimiento.



2.4 Tecnología elegida

Tabla 4. Tecnologías del proyecto.

Capa	Tecnología	Por qué se usa
Backend	Python 3.11, Django (requirements pide 5.0 o superior), Django REST Framework 3.15	ORM con consultas parametrizadas, hash PBKDF2 de contraseñas, validadores y middleware de seguridad incluidos.
Base de datos	PostgreSQL 16 con psycopg 3	Transacciones y bloqueo de filas (select_for_update) para pagos y stock.
Frontend	Vue 3.5	Las plantillas de Vue escapan HTML por defecto y el build estático es fácil de servir.
Pagos	Transbank SDK (Webpay Plus)	Los datos de tarjeta se ingresan en Transbank y nunca pasan por el servidor.
Autenticación	JWT firmado con HS256 (PyJWT)	API sin estado para la SPA; se va a agregar renovación y revocación.
Operación	GitHub y GitHub Projects	Trazabilidad entre tareas, commits y PR.

3. Proceso DevOps y DevSecOps del equipo

Se definió un ciclo de ocho fases que se repite en cada sprint semanal (figura 1). En esta etapa queda planificado; su aplicación se muestra en las etapas siguientes. Para cada fase se definió una práctica de seguridad aplicada a este proyecto y la evidencia con la que se va a mostrar. Las prácticas se basan en OWASP (ASVS 4.0.3, Top 10 2021 y WSTG), NIST SP 800-218 (SSDF 1.1) e ISO/IEC 27001:2022.

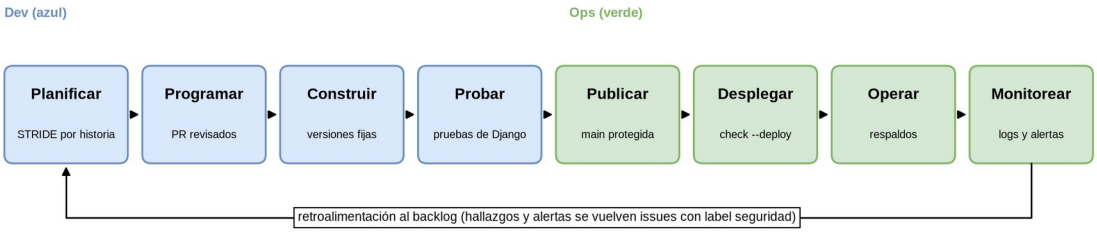


Figura 1. Ciclo DevSecOps del equipo con la práctica principal de cada fase.

Tabla 5. Fases del ciclo, práctica de seguridad y evidencia.

Fase	Práctica de seguridad en el proyecto	Evidencia
Planificar	Cada historia nueva pasa por STRIDE sobre el diagrama de la sección 5. Las amenazas se transforman en criterios de aceptación y en tareas con label seguridad.	GitHub Projects; matriz amenaza a requerimiento.
Programar	Ramas feature/ desde develop y PR revisados. Revisión obligatoria del Security Champion en apps/accounts y apps/payments.	PR revisados y enlazados a su issue.
Construir	Versiones exactas en requirements.txt.	Log del job de dependencias.
Probar	Pruebas de seguridad en Django para roles, sesiones y validación (pasar de 8 a 30 o más).	Resultado de las pruebas de Django en cada PR.
Publicar	main protegida: solo entra código por PR aprobado desde QA, con todos los checks en verde. Cada versión se marca con un tag y notas con los hallazgos corregidos.	Regla de protección; releases en GitHub.
Desplegar	Settings por entorno con DEBUG=False, HSTS y cookies seguras; gunicorn detrás de proxy TLS; secretos en variables de entorno o GitHub Secrets; PostgreSQL sin puerto publicado y con usuario de privilegios mínimos.	Salida de check --deploy; settings de producción en el repositorio.
Operar	Respaldo periódico de la base de datos.	Registro de los respaldos realizados.
Monitorear	LOGGING en JSON para login, 403, 429, cambios de rol y pagos, con la IP tomada solo del proxy de confianza. Alerta con más de 3 logins fallidos en 5 minutos. Revisión de logs en cada cierre de sprint.	Extracto de logs; alerta disparada en una prueba.

4. Requerimientos

4.1 Requerimientos funcionales (historias de usuario)

Estas historias describen el producto que se espera tener al final del proyecto. Su implementación no es parte del plan de esta etapa.

En la figura 2 se muestran los casos de uso del sistema, asociados a los actores de la tabla 3 y a las historias de la tabla 6.

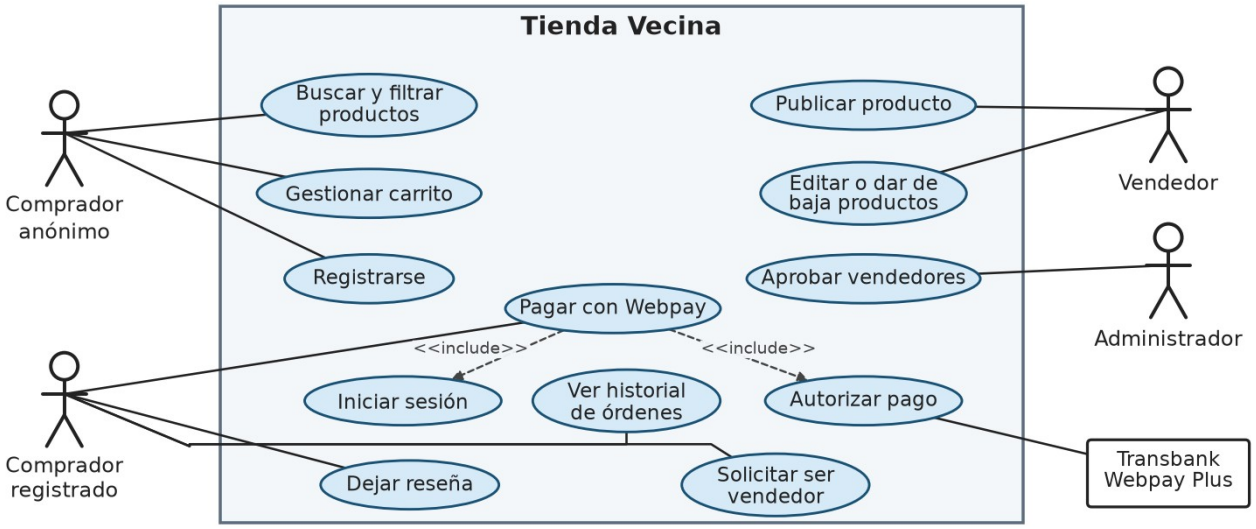


Figura 2. Diagrama de casos de uso.

Tabla 6. Historias de usuario con criterios de aceptación.

ID	Historia	Criterios de aceptación (verificables)	Prioridad
HU01	Como comprador quiero registrarme con correo y contraseña para poder comprar	1) Contraseña con menos de 12 caracteres, común o solo numérica responde 400 con mensaje claro. 2) Correo duplicado responde con mensaje que no confirma la existencia de la cuenta. 3) Toda cuenta creada por registro público queda con rol BUYER aunque el cuerpo envíe role=SELLER. 4) Pruebas automatizadas cubren los tres casos.	Alta
HU02	Como usuario quiero iniciar sesión de forma segura para acceder a mi cuenta	1) Más de 3 intentos por minuto desde la misma IP responden 429. 2) Usuario inexistente, contraseña incorrecta y cuenta bloqueada responden el mismo mensaje y código. 3) Un atacante no puede dejar bloqueada indefinidamente una cuenta ajena (bloqueo progresivo por cuenta e IP). 4) Login exitoso entrega token y registra evento de auditoría.	Alta
HU03	Como usuario quiero que mi sesión expire y poder cerrarla para que nadie la reutilice	1) Token de acceso con vida de 15 minutos. 2) Token de renovación rotativo de 7 días, invalidado al usarse. 3) Al cerrar sesión, el token de renovación queda revocado y un nuevo uso responde 401.	Alta
HU04	Como administrador quiero aprobar a los vendedores para que solo publiquen los vendedores que estén verificados	1) Un usuario puede solicitar ser vendedor y su solicitud queda PENDIENTE. 2) Solo un administrador aprueba o rechaza. 3) Un vendedor no aprobado recibe 403 al publicar. 4) El cambio de rol queda registrado en el log de auditoría.	Media
HU05	Como administrador quiero un panel /admin protegido para que no sea un punto de entrada	1) La ruta del panel se define por variable de entorno. 2) En producción solo responde desde la red permitida (proxy o VPN). 3) Intentos fallidos de acceso al panel quedan en el log y se limitan.	Media



ID	Historia	Criterios de aceptación (verificables)	Prioridad
HU06	Como vendedor quiero publicar un producto con foto y descripción para venderlo	1) Imagen JPG, PNG o WebP de hasta 5 MB; otro tipo o tamaño responde 400. 2) La imagen se guarda normalizada a 900x600 y 1 MB como máximo. 3) Título, ADS y descripción se validan con lista blanca de caracteres y largo máximo. 4) Precio mayor que 0 y menor que 100.000.000 CLP.	Media
HU07	Como vendedor quiero editar o eliminar solo mis productos	1) El dueño puede editar y eliminar su producto. 2) Otro vendedor recibe 404 al intentar modificarlo. 3) Un comprador recibe 403. 4) Pruebas de IDOR en el pipeline.	Media
HU08	Como comprador quiero buscar productos por texto y categoría para encontrar lo que necesito	1) Resultados paginados con un máximo de 50 por página. 2) Parámetro de búsqueda de máximo 100 caracteres; uno más largo responde 400. 3) p95 menor a 800 ms con 1.000 productos y 50 usuarios simultáneos.	Baja
HU09	Como comprador quiero ver el total de mi carrito calculado por el servidor	1) quantity debe ser entero entre 1 y 10 y no mayor al stock; otro valor responde 400, nunca 500. 2) payment_method solo acepta valores de la lista definida. 3) El total coincide con los precios de la base de datos aunque el cliente envíe precios. 4) Errores no exponen detalles internos.	Alta
HU10	Como comprador quiero pagar con Webpay Plus y recibir mi comprobante	1) Una orden solo pasa a AUTHORIZED si Transbank responde response_code 0 y el monto coincide con total_amount. 2) Si Transbank no responde o lanza error, la orden queda FAILED. 3) Un segundo commit del mismo token no cambia la orden ni el stock. 4) El stock se descuenta dentro de transaction.atomic con bloqueo de fila y nunca queda negativo. 5) El modo simulado solo existe con TRANSBANK_ENVIRONMENT=MOCK y DEBUG activo.	Alta
HU11	Como comprador quiero consultar el estado de mis órdenes	1) GET de órdenes devuelve solo las del usuario autenticado. 2) Pedir una orden ajena responde 404. 3) La respuesta no incluye datos de tarjeta salvo los últimos 4 dígitos.	Baja

4.2 Requerimientos no funcionales

Tabla 7. Requerimientos no funcionales medibles.

ID	Requerimiento	Meta medible	Cómo se mide
RNF01	Rendimiento del catálogo	p95 menor a 800 ms en listado y búsqueda con 50 usuarios simultáneos y 1.000 productos	Prueba de carga
RNF02	Rendimiento del pago	Creación de la transacción en menos de 2 s (p95), sin contar la latencia de Transbank	Duración registrada por el middleware de auditoría
RNF03	Disponibilidad	99% de chequeos de salud exitosos cada 5 minutos en las semanas 8 y 9	Monitor sobre /health
RNF04	Mantenibilidad	Cobertura de 70% o más en apps/accounts y apps/payments	coverage.py en el pipeline
RNF05	Calidad de código	0 hallazgos de severidad alta abiertos al cierre de cada sprint desde el Sprint 4	Análisis estático del código en el pipeline
RNF06	Corrección de vulnerabilidades	Críticas en 7 días o menos y altas en 14 días o menos desde que se registran	Fechas de los issues con label hallazgo
RNF07	Respaldo	Pérdida máxima de 24 h de datos (RPO) y restauración en menos de 30 min	Prueba de restauración registrada
RNF08	Usabilidad	Del carrito al comprobante en 4 pantallas o menos	Prueba con 3 usuarios externos al equipo

4.3 Requerimientos de seguridad

Cada requerimiento de seguridad indica cómo se va a verificar. La relación con las amenazas de la sección 5.3 está en la tabla 11.

Tabla 8. Requerimientos de seguridad y su verificación.

ID	Requerimiento	Verificación
RS01	Una orden pasa a pagada solo si Transbank confirma con response_code 0 y el monto coincide con el total de la orden. Cualquier error deja la orden FAILED. El modo simulado existe solo con TRANSBANK_ENVIRONMENT=MOCK.	Pruebas con Transbank simulado que falla, que responde código distinto de 0 o monto distinto: las tres dejan la orden FAILED.



ID	Requerimiento	Verificación
RS02	La confirmación del pago es idempotente y el descuento de stock es atómico (transaction.atomic y select_for_update).	Prueba con dos confirmaciones del mismo token: una sola aprobada, stock descontado una vez y nunca negativo.
RS03	Ningún secreto en el código ni en el historial. La app no arranca si falta SECRET_KEY, JWT_SECRET_KEY o la llave de Transbank fuera del modo prueba. Las llaves expuestas se rotan.	gitleaks en el pipeline y sobre el historial con 0 hallazgos; prueba de arranque sin la variable.
RS04	Contraseñas de 12 caracteres o más, revisadas con validate_password (comunes, solo numéricas o parecidas al correo).	Prueba unitaria de registro con contraseñas débiles: responde 400.
RS05	Login y registro limitados a 3 intentos por minuto por IP (429). Mismo mensaje para usuario inexistente, clave incorrecta y cuenta bloqueada. Bloqueo progresivo que no deja cuentas ajenas bloqueadas indefinidamente.	Script de 10 intentos; comparación de respuestas; prueba unitaria del 429.
RS06	El registro público crea solo compradores. Pasar a vendedor (incluido upgrade-seller) deja una solicitud pendiente; publicar exige ser vendedor aprobado por un administrador.	Prueba de registro con role=SELLER (queda BUYER), de upgrade-seller sin aprobación y de publicación sin aprobación (403).
RS07	Autorización por objeto: un vendedor modifica solo sus productos y un comprador ve solo sus órdenes.	Pruebas de IDOR con dos usuarios del mismo rol.
RS08	Token de acceso de 15 minutos (hoy dura 7 días), renovación rotativa de 7 días que se revoca al cerrar sesión, y tokens en cookie HttpOnly, Secure y SameSite=Strict en vez de localStorage.	Prueba de uso del token después de cerrar sesión (401); revisión en DevTools.
RS09	Validación en el servidor con lista blanca, tipo y rango: quantity entre 1 y 10, IDs como UUID, payment_method dentro de la lista, textos con largo máximo y paginación de 50. Reemplaza las expresiones de lista negra.	Pruebas parametrizadas con valores inválidos: siempre 400, nunca 500.
RS10	Hasta 5 fotos por producto, cada una de 5 MB como máximo y 15 MB por solicitud, tipo verificado por contenido y normalizadas a 900x600. Las URLs de fotos solo pueden apuntar al media/ del sistema.	Pruebas con extensión falsa, archivo políglota, 6 fotos, archivo de 10 MB y URL externa.
RS11	Producción con DEBUG=False, HSTS, cookies seguras, CSP con connect-src limitado a la API, gunicorn, sin cambio automático a SQLite y sin cuentas de demostración.	manage.py check --deploy sin advertencias; cabeceras con curl -I.
RS12	PostgreSQL con usuario de aplicación sin privilegios de superusuario, sin puerto publicado y con respaldo diario.	Consulta de privilegios del rol; escaneo de puertos del host; restauración de prueba.
RS13	Registro de eventos de seguridad (login, 403, 429, cambios de rol, pagos) con fecha, usuario e IP real, sin contraseñas, tokens ni datos de tarjeta.	Prueba que provoca un login fallido y revisa la línea de log.
RS14	Errores con mensaje genérico e ID de correlación; el detalle queda solo en el log.	Prueba que fuerza una excepción y revisa la respuesta.
RS15	Panel /admin en ruta configurable, accesible solo desde la red permitida y con intentos limitados.	Acceso desde fuera de la red permitida (403) y a la ruta por defecto (404).
RS16	Dependencias con versión fija y sin vulnerabilidades altas o críticas conocidas.	pip-audit y npm audit en el pipeline.
RS17	La insignia de tienda oficial la otorga un administrador después de verificar al vendedor. La URL externa debe ser https, se valida al guardarla y se abre con aviso de sitio externo y rel=noopener.	Prueba de register-store sin aprobación (no queda oficial) y con URL javascript: o http (400).

5. Amenazas y diseño

5.1 Método

Se aplicó STRIDE por elemento sobre el diagrama de la figura 3, poniendo atención en los flujos que cruzan un límite de confianza. Cada amenaza sale de la revisión del código del proyecto y se clasifica por riesgo según probabilidad e impacto. Los hallazgos todavía no están corregidos. En esta etapa quedan como requerimientos de seguridad; las tareas para corregirlos se planificarán al comenzar las etapas siguientes.

5.2 Arquitectura, datos y límites de confianza

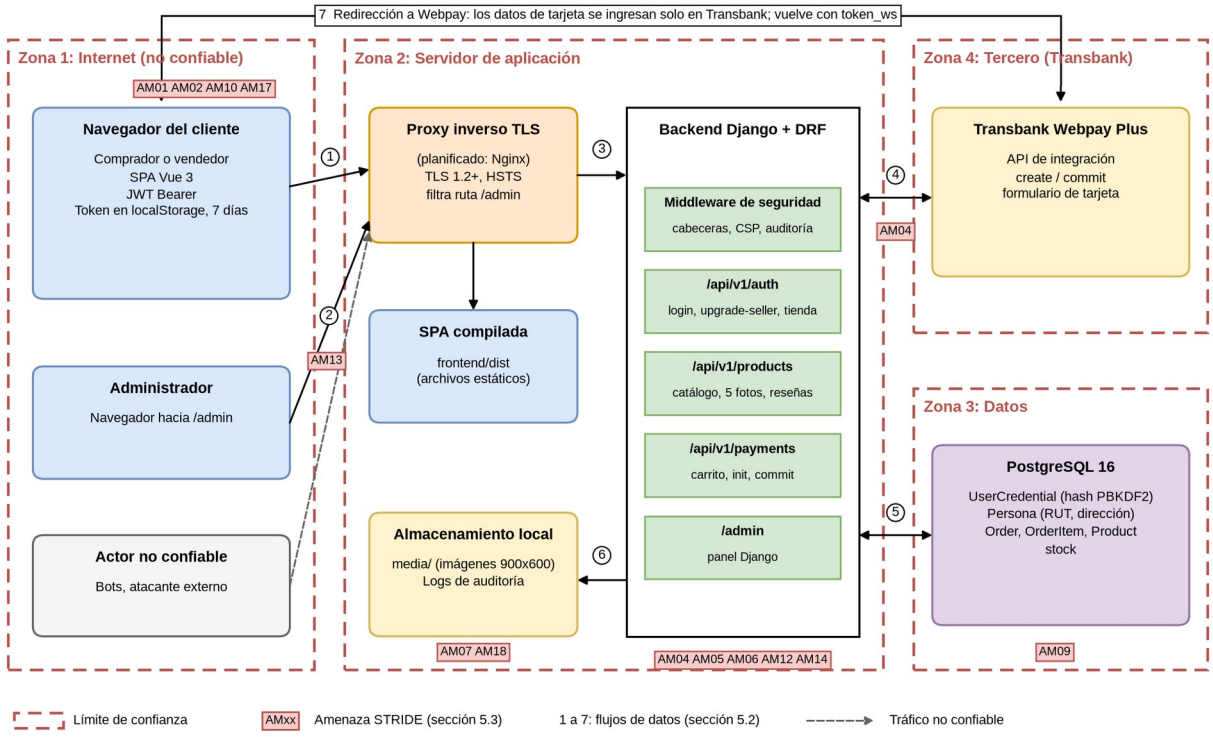


Figura 3. Arquitectura de Tienda Vecina con zonas de confianza, flujos de datos y amenazas.

Tabla 9. Flujos de datos del diagrama.

N.º	Flujo	Datos	Límite que cruza	Control planificado
1	Navegador a proxy	Credenciales, JWT, carrito (IDs y cantidades), imágenes	Internet a servidor	TLS, límite de intentos, validación con lista blanca
2	Administrador a /admin	Credenciales de staff, sesión de Django	Internet a servidor	Ruta configurable, restricción de origen
3	Proxy a backend	Mismo tráfico y X-Forwarded-For	Interno	IP real tomada solo desde el proxy
4	Backend y Transbank	Orden, monto, código de comercio y llave; respuesta de autorización	Servidor a tercero	HTTPS, llave en variable de entorno, validar monto y código
5	Backend y PostgreSQL	Hash de contraseñas, Persona (RUT, dirección), órdenes, stock	Aplicación a datos	ORM parametrizado, usuario mínimo, red interna
6	Backend a almacenamiento	Imágenes normalizadas, logs de auditoría	Interno	Permisos de archivo, logs sin datos sensibles
7	Navegador y Webpay	Datos de tarjeta (solo en Transbank); token_ws al volver	Internet a tercero	El servidor nunca recibe datos de tarjeta

5.3 Amenazas STRIDE del sistema

Tabla 10. Amenazas STRIDE identificadas en el código.

ID	STRIDE	Amenaza en el sistema	Evidencia en el código	Riesgo	Mitiga
AM01	S	Fuerza bruta o relleno de credenciales contra /api/v1/auth/login/: no hay throttling de DRF.	accounts/views.py; settings sin DEFAULT_THROTTLE_RATES	Alto	RS04, RS05
AM02	S	Robo de sesión: un XSS o una extensión lee el JWT desde localStorage y lo usa hasta 7 días (el PR #2 subió la vida del token de 24 h a 168 h); no hay renovación ni revocación.	stores/auth.js; settings.py JWT_EXPIRATION_HOURS=168 (develop)	Alto	RS08, RS09



ID	STRIDE	Amenaza en el sistema	Evidencia en el código	Riesgo	Mitiga
AM03	S, E	Tokens falsificados: si se despliega con el SECRET_KEY o JWT_SECRET_KEY de las plantillas versionadas, cualquiera puede firmar un JWT de administrador.	example.env, backend/example.env, .env.example	Alto	RS03
AM04	T	Pago aprobado sin pagar: si Transbank falla o el token empieza con tbk_token_, commit_transaction devuelve AUTHORIZED.	payments/services.py:120-146	Crítico	RS01
AM05	T	Manipulación de parámetros: int(quantity) sin rango produce error 500 con texto interno; payment_method no se valida.	payments/services.py; payments/views.py	Medio	RS09, RS14
AM06	T	Repetición y carrera en el commit público: el mismo token se confirma varias veces sin revisar el monto y el stock se descuenta fuera de una transacción (sobreventa).	payments/views.py:127-153	Alto	RS01, RS02
AM07	R	Repudio: sin LOGGING configurado los eventos de security.audit se pierden, y la IP sale de X-Forwarded-For sin validar.	config/settings.py; security/middleware.py:46	Medio	RS13
AM08	I	Fuga de información: errores que devuelven str(e), print() de excepciones y DEBUG=True en las plantillas.	payments/views.py; payments/services.py; example.env	Medio	RS11, RS14
AM09	I	Si PostgreSQL no responde, la app cambia a SQLite (opción activa por defecto) y deja RUT y direcciones en un archivo local sin control.	config/settings.py:103-121	Medio	RS11, RS12
AM10	D	Bloqueo de cuentas ajenas: 5 intentos fallidos bloquean 15 minutos cualquier correo, y el mensaje distinto de cuenta bloqueada confirma que la cuenta existe.	accounts/serializers.py:152-161	Medio	RS05
AM11	D	Agotamiento de recursos: el listado de productos (APIView) no pagina y la subida de archivos no tiene límite de tamaño.	products/views.py; settings	Medio	RS09, RS10
AM12	E	Escalada a vendedor: el registro público acepta role=SELLER y, desde el PR #1, POST /api/v1/auth/upgrade-seller/ cambia el rol a vendedor en un clic sin aprobación.	accounts/serializers.py:63-66; accounts/views.py UpgradeToSellerAPIView	Alto	RS06
AM13	E	Panel /admin en la ruta por defecto, sin restricción de origen ni límite de intentos.	config/urls.py:11	Medio	RS15
AM14	E	Acceso a objetos ajenos (IDOR) al editar productos o consultar órdenes si se valida solo el rol.	products/views.py; endpoint de órdenes planificado	Medio	RS07
AM15	T	Cadena de suministro: requirements.txt usa >= y puede instalar una versión vulnerable sin que nadie lo note.	backend/requirements.txt	Medio	RS16
AM16	S, E	Cuenta administradora con contraseña pública: el comando de carga de datos de prueba corre en cada arranque y el README publica las claves de demostración.	README.md	Alto	RS11
AM17	S	Suplantación de marcas: cualquier vendedor se declara tienda oficial con nombre y URL externa propios (PR #2), y la insignia lleva a esa URL, lo que sirve para phishing.	accounts/views.py RegisterStoreAPIView	Medio	RS17
AM18	D, T	Carga de hasta 5 fotos por producto en una sola solicitud (PR #3) sin límite de tamaño total, y campo images que acepta URLs que podrían apuntar fuera del sitio.	products/models.py images (JSONField); products/serializers.py	Medio	RS10

También se registraron como hallazgos otras debilidades menores: CSP con connect-src que acepta cualquier origen https, validación de textos con expresiones de lista negra (products/serializers.py) y una base de solo 8 pruebas.

5.4 Trazabilidad amenaza, requerimiento, historia y prueba

Tabla 11. Trazabilidad entre amenazas, requerimientos, historias y pruebas.

Amenazas	Requerimientos	Historias (Proceso: config. del repo o despliegue)	Prueba o evidencia planificada
AM04, AM06	RS01, RS02	HU10	test_commit_falla_si_transbank_no_responde; test_doble_commit_no_descuenta_stock
AM05, AM11, AM18	RS09, RS10	HU06, HU08, HU09	test_cantidad_invalida_responde_400; test_imagen_extension_falsa
AM03, AM16	RS03, RS11	Proceso	Job gitleaks; check --deploy; arranque sin SECRET_KEY falla
AM01, AM10	RS04, RS05	HU01, HU02	test_login_429; test_mensajes_login_identicos



Amenazas	Requerimientos	Historias (Proceso: config. del repo o despliegue)	Prueba o evidencia planificada
AM02	RS08	HU03	test_refresh_revocado_tras_logout; revisión en DevTools
AM12	RS06	HU01, HU04	test_registro_ignora_rol_seller; test_publicar_sin_aprobacion_403
AM14	RS07	HU07, HU11	test_vendedor_no_edita_producto_ajeno; test_orden_ajena_404
AM07	RS13	Proceso	test_login_fallido_genera_log; alerta de prueba
AM08, AM09	RS11, RS14	HU09	test_error_generico_con_id; check --deploy
AM09	RS12	Proceso	Consulta de privilegios del rol; nmap al host
AM13	RS15	HU05	Acceso externo a /admin responde 403
AM15	RS16	Proceso	Jobs pip-audit y npm audit
AM17	RS17	HU04	test_tienda_oficial_requiere_aprobacion; test_url_tienda_invalida_400

6. Planificación en GitHub Projects

Acordado con el Product Owner, el equipo utiliza GitHub Projects en lugar de Azure Boards para la planificación. Se pidió porque el código ya está en GitHub y así las tareas, las ramas y los PR quedan en el mismo lugar, sin conectar dos plataformas. Se armó la jerarquía de épicas, features, historias y tareas con issues: las épicas y features son issues con label (epica o feature) y funcionan como issue padre; las historias son sub-issues de su feature y las tareas son sub-issues de su historia. Los sprints se manejan con el campo Iteration y los responsables con los assignees de cada issue. El plan cubre solo la primera etapa (semanas 1 a 3); lo que viene después se cargará al empezar cada etapa. El Product Owner prioriza y acepta las historias. El Product Owner lo autorizó de palabra.

6.1 Configuración

- Project: Tienda Vecina - Planificación DevSecOps (https://github.com/users/RobertxDessinacap/projects/2), vinculado al repositorio Tienda_Vecina_INACAP.
- Campo Iteration Sprint, de una semana cada una: Sprint 1 desde el 19-10-2026, Sprint 2 desde el 26-10-2026 y Sprint 3 desde el 02-11-2026.
- Campo Status con las columnas Backlog, Ready, In progress, In review y Done. Al cierre de este informe hay 12 ítems en Done (todo lo de los sprints 1 y 2 y la protección de ramas), 41 en In progress (las épicas, las features y la mayoría del Sprint 3) y 10 tareas del Sprint 3 en Ready, como las capturas, la plantilla de PR, los issues de los PR #1 a #4 y la publicación del informe.
- Labels epica, feature, historia, tarea y seguridad, entre otros, y estimación en horas en cada tarea.
- El backlog se cargó desde Backlog_GitHub_Projects_v2.csv: épicas #5 a #10, features #11 a #20, historias #21 a #32 y tareas #33 a #67, unidas como sub-issues según la columna Parent.

6.2 Estructura del backlog

Tabla 12. Épicas del backlog de la primera etapa.

Épica (issue con label epica; EP5 registra los PR #1 a #4)	Features	Historias	Tareas	Tareas de seguridad
EP1 Organización del equipo (#5)	2	2	6	0
EP2 Requerimientos (#6)	2	2	7	2
EP3 Amenazas y proceso (#7)	2	2	5	5
EP4 Repositorio GitHub (#8)	2	2	8	3
EP5 Funciones del periodo (#9)	1	3	4	1
EP6 Informe de planificación (#10)	1	1	5	0
Total	**10**	**12**	**35**	**11**

6.3 Sprints, objetivos y responsables

Tabla 13. Sprints de la primera etapa con sus integrantes y horas (campo Iteration).

Sprint	Semana	Objetivo	Historias (issues)	Tareas	Horas	Integrantes
Sprint 1 Done	Semana 1 19 al 25 oct	Organizar el equipo, leer la documentación y dejar el entorno local funcionando.	#21, #23	4 (ninguna de seguridad)	8 h	Díaz 6 h, Ancán 2 h
Sprint 2 Done	Semana 2 26 oct al 1 nov	Revisar la seguridad del código y definir los requerimientos funcionales, no funcionales y de seguridad.	#23, #24	4 (2 de seguridad)	14 h	Sanhueza 8 h, Yáñez 6 h
Sprint 3 En curso	Semana 3 2 al 8 nov	Modelo de amenazas y arquitectura, proceso DevSecOps, repositorio y tablero, registro de los PR #1 a #4 e informe de planificación.	#22, #24, #25, #26, #27, #28, #29, #30, #31, #32	27 (9 de seguridad)	53 h	Ancán 17 h, Sanhueza 15 h, Yáñez 13 h, Díaz 8 h

El backlog tiene 35 tareas, 11 de ellas con el label seguridad. Las horas totales por integrante quedan así: Sanhueza 23 h, Ancán 19 h, Yáñez 19 h y Díaz 14 h. La semana 3 es la que tiene más carga porque junta el modelo de amenazas, la configuración del repositorio y el informe, y en ella trabajan los cuatro. El detalle de quién tiene cada tarea está en el anexo A.

6.4 Evidencia en GitHub Projects

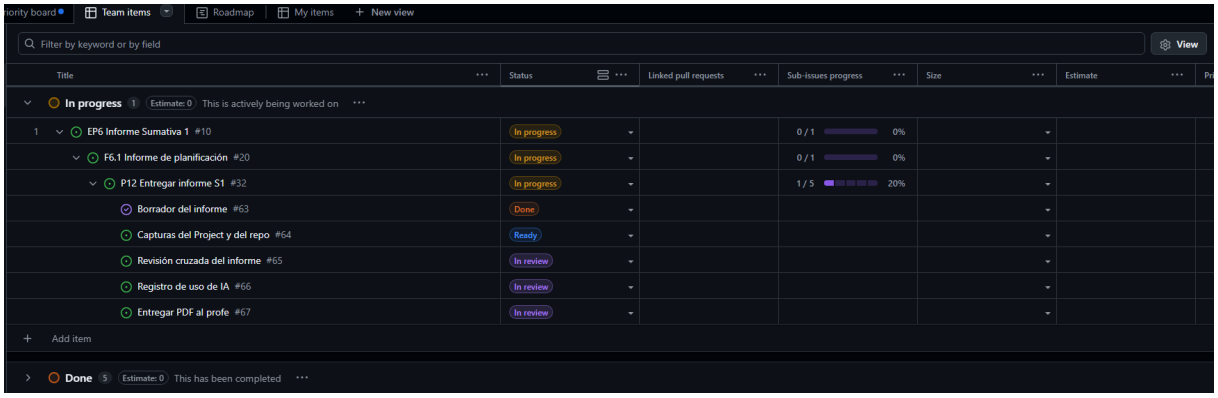


Figura 4. Backlog en vista tabla con la jerarquía de épica, feature, historia y tareas.

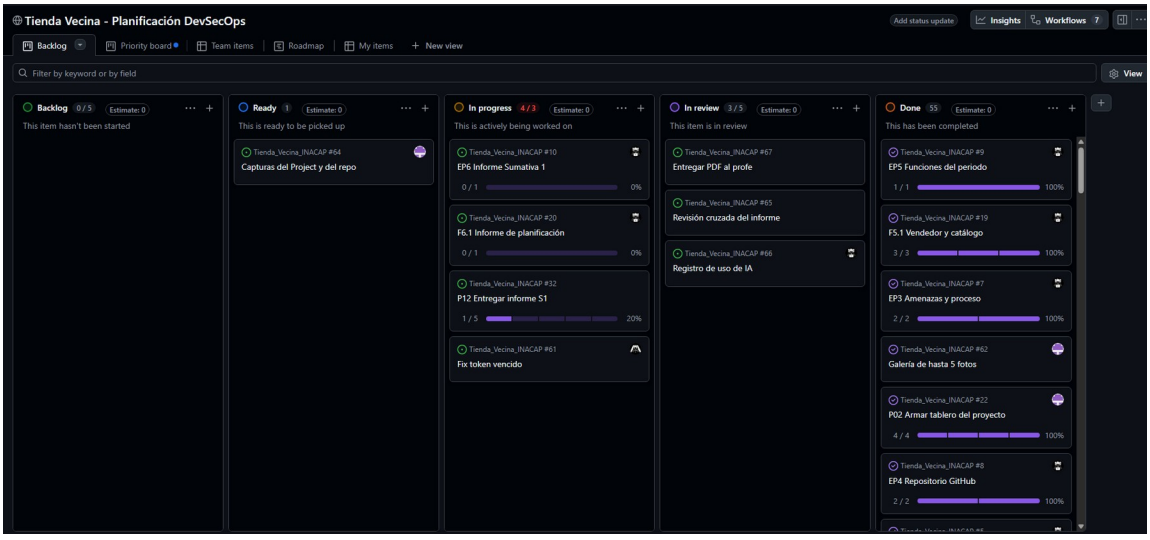


Figura 5. Tablero por Status con responsables.

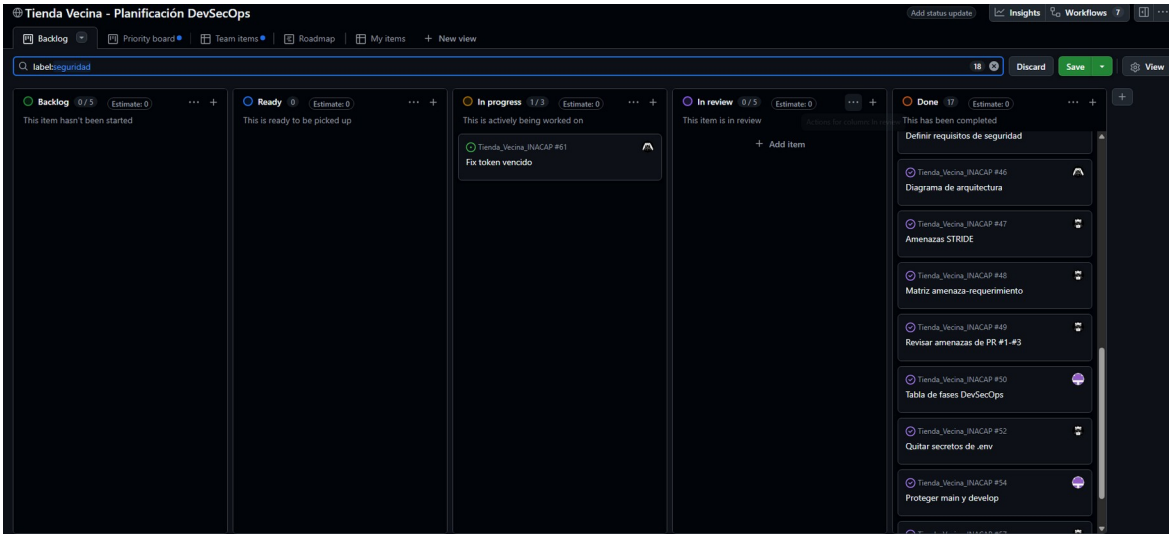


Figura 6. Tablero filtrado por label:seguridad.

7. Repositorio en GitHub

El repositorio es https://github.com/RobertxDessinacap/Tienda_Vecina_INACAP. El dueño es RobertxDessinacap y Mjancanes también aporta código.

7.1 Estructura y ramas

En la raíz están backend/ (Django y DRF con las apps accounts, products, payments y security), frontend/ (Vue 3), la plantilla .env.example y los archivos .gitignore, README.md, DOCUMENTACION_SISTEMA.md y GUIA_TRANSBANK_WEBPAY.md. La carpeta docs/ se va a crear para dejar los informes; el PDF de este informe se sube a docs/S1_informe_planificacion.pdf.

Tabla 14. Ramas del repositorio.

Rama	Uso
main	Versión estable que se publica.
QA	Integración para revisar y probar antes de pasar a main.
develop	Rama de desarrollo donde se integran las features.
feature/seller-modal-and-price-filter	Modal de vendedor y filtro de precio (PR #1, ya integrada).
feature/seller-panel-and-official-store	Panel de vendedor, tienda oficial y reseñas (PR #2, ya integrada).
feature/multi-photo-gallery-and-ui-cleanup	Galería de 5 fotos y limpieza de rutas técnicas (PR #3, ya integrada).

7.2 Pull requests del periodo

Tabla 15. PR integrados en el periodo.

PR	Título	Autor	Fecha	Estado
#1	feat: Flujo de vendedor, filtro dual de precio y paleta centralizada	RobertxDessinacap	4 nov 2026	Merged a develop
#2	feat: panel vendedor, edicion/baja de publicaciones, tienda oficial, reviews y fix de sesion expirada	RobertxDessinacap	5 nov 2026	Merged a develop; reporta 10/10 pruebas de backend
#3	feat: soporte de hasta 5 fotos por producto, confirmacion de eliminacion y limpieza de rutas tecnicas	Mjancanes (merge de RobertxDessinacap)	5 nov 2026	Merged a develop; reporta 10/10 pruebas de backend
#4	refactor(branding): actualizar nombre e identidad del proyecto a Tienda Vecina	RobertxDessinacap	9 nov 2026	Merged a develop
#68	Cambios importantes (corrección menor del título del README)	Mjancanes	10 nov 2026	Merged a main

El flujo que se ve en el repositorio es feature hacia develop, luego develop hacia QA y finalmente QA hacia main. La excepción es el PR #68, una corrección menor del título del README que entró directo a main. Hay que reconocer una brecha: los PR #1 a #4 no están enlazados a issues, porque el tablero todavía no existía cuando se hicieron, y tampoco tienen una revisión formal registrada. Los issues retroactivos se registran en la tarea #58 y desde ahora cada cambio va a partir de un issue y a cerrarlo desde el PR.

7.3 README, .gitignore y secretos

El .gitignore ya excluye .env y .env.*, llaves (*.pem, *.key), db.sqlite3, media/, node_modules/ y dist/. El problema está en las plantillas: example.env, backend/example.env y los .env.example traen SECRET_KEY y JWT_SECRET_KEY con valor. En el Sprint 3 se van a dejar solo marcadores en un único .env.example y rotar las llaves. Al README se le va a agregar el flujo de ramas y los enlaces, y se van a sacar las contraseñas de demostración.

7.4 Reglas planificadas

main, QA y develop ya están protegidas con Rulesets en GitHub (Protección MAIN, Protección QA y Protección DEVELOP), con lo que la tarea #54 queda hecha. El resto de las reglas se planifica para el Sprint 3 (historias #27 y #28):

- Como objetivo, en las tres ramas no se permite push directo ni force push y todo cambio entra por PR con al menos 1 aprobación de otro integrante.
- Ramas cortas desde develop con nombre feature/descripcion o fix/descripcion, y borrado de la rama después del merge.
- Cada commit y PR referencia su issue con #n, por ejemplo "Reemplaza secretos de las plantillas .env (#52)", y la descripción del PR usa closes #52 para que el issue se cierre solo al hacer merge y pase a Done en el Project.
- Revisión obligatoria del Security Champion en los PR que tocan accounts y payments.

7.5 Evidencia en GitHub

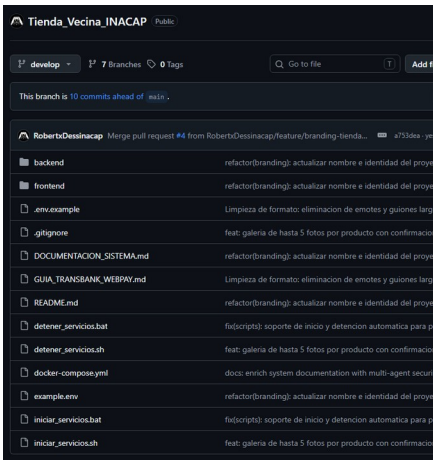


Figura 7. Repositorio en la rama develop.

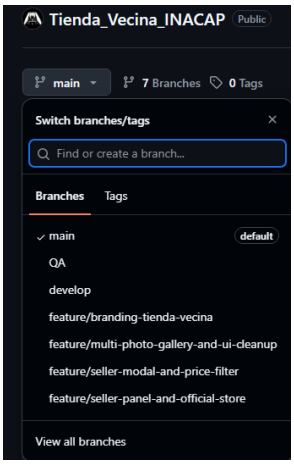


Figura 8. Ramas del repositorio.

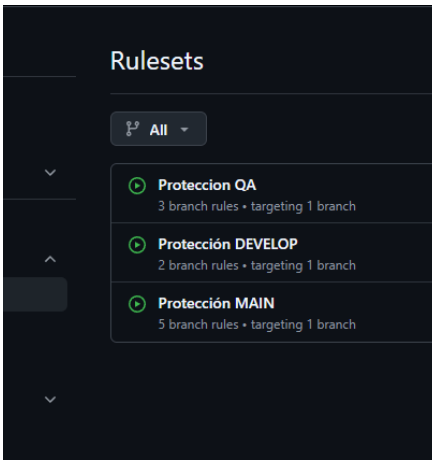


Figura 9. Rulesets de main, QA y develop.

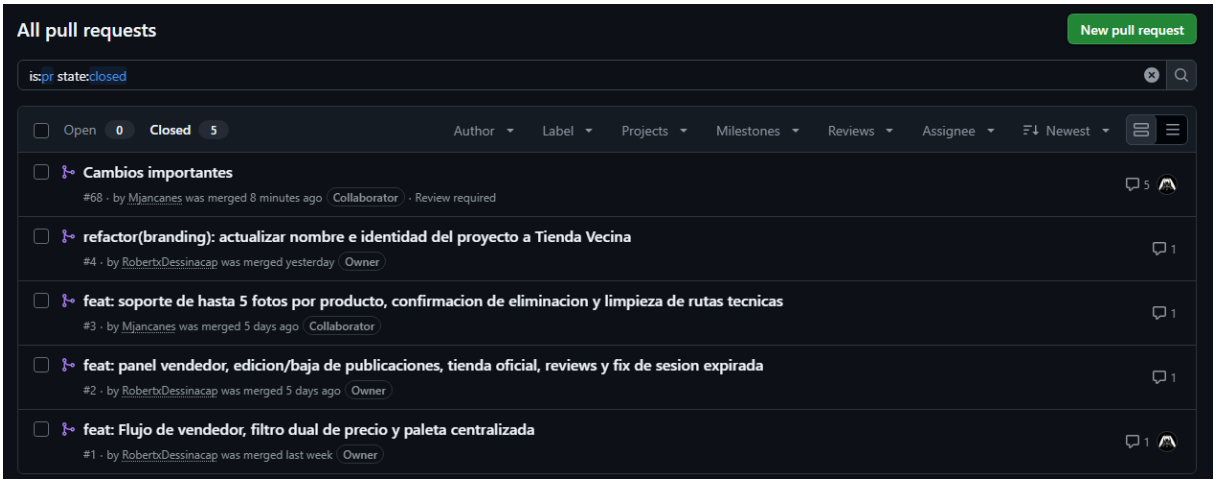


Figura 10. PR integrados en el periodo.

8. Cronograma y riesgos

8.1 Hitos de la primera etapa

Tabla 16. Hitos hasta el cierre de la etapa (10 de noviembre).

Semana	Hito	Resultado o evidencia	Responsable
1 (Sprint 1)	H1: equipo organizado	Roles y responsabilidades; entorno local funcionando	Díaz
2 (26 oct al 1 nov)	H2: requerimientos	Historias de usuario, requerimientos no funcionales y requerimientos de seguridad; lista de hallazgos del código	Yáñez
3 (2 al 8 nov)	H3: diseño y herramientas	Modelo STRIDE, diagrama, proceso DevSecOps, Project con el backlog, repositorio limpio y con reglas	Sanhueza
3 (2 al 8 nov)	H4: incremento del periodo	PR #1 a #4 integrados a develop y enlazados a sus issues	Yáñez, Ancán
Cierre 10 nov	H5: informe de planificación publicado	El informe se sube a docs/S1_informe_planificacion.pdf y acceso del Product Owner al repositorio y al Project	Díaz, Yáñez

8.2 Riesgos de esta etapa

Tabla 17. Riesgos técnicos y no técnicos con su mitigación.

ID	Riesgo	Tipo	Prob. / Impacto	Mitigación	Responsable
R1	Poco tiempo antes del 10 de noviembre por otros trabajos en paralelo	No técnico	Alta / Alto	Tareas de máximo 4 h, revisión de avance a mitad de semana y reasignación en el Project	Díaz
R2	El cambio a GitHub Projects no queda respaldado	No técnico	Baja / Alto	Dejar registrado el acuerdo con el Product Owner y explicarlo en la sección 6	Díaz
R3	El uso de IA genera código o controles incorrectos	No técnico	Media / Medio	Todo aporte de IA se revisa contra el código y queda en el registro de la sección 9	Yáñez
R4	Las plantillas .env con llaves quedan en el historial del repositorio	Técnico	Alta / Alto	Reemplazar por marcadores, rotar las llaves y no usar esos valores en ningún despliegue	Sanhueza
R5	Se integran cambios a develop sin revisión ni issue, como pasó con los PR #1 a #4	Técnico	Media / Medio	Mantener la protección de main, QA y develop y crear los issues retroactivos en el Sprint 3	Ancán
R6	Las funciones nuevas agregan amenazas que no se alcanzan a modelar	Técnico	Media / Medio	Revisar cada PR nuevo contra el diagrama y la tabla STRIDE antes del cierre de la etapa	Sanhueza



9. Conclusiones y uso de IA

9.1 Aprendizajes

Lo primero que se constató es que la documentación no prueba nada por sí sola. El README habla de ISO 27001 y OWASP, pero en el código hay un pago que se aprueba sin Transbank, secretos en archivos versionados y un registro donde cualquiera se declara vendedor. Por eso cada requerimiento de seguridad tiene una forma de verificarlo.

También se vio que el código avanza más rápido que la planificación si no hay orden. Los PR #1 a #4 agregaron funciones útiles, pero los PR #1 a #3 también dejaron dos amenazas nuevas (AM17 y AM18), alargaron la vida del token a 7 días y quedaron sin issue que los respalde. Por eso el tablero en GitHub Projects y la regla de partir cada cambio desde un issue son lo primero que hay que cumplir. La trazabilidad entre amenaza, requerimiento e historia sirve para planificar las etapas siguientes.

La revisión y las pruebas de esta etapa se hicieron sobre el mockup funcional que se mostró al Product Owner, así que los hallazgos corresponden a esa primera versión y no a un sistema en producción.

9.2 Dificultades

Resultó difícil separar lo que corresponde a esta etapa de lo que viene después y coordinar horarios entre los cuatro. Otra dificultad fue la poca información sobre los requerimientos por parte del Product Owner, por lo que varios se definieron a partir del código y la documentación existentes.

9.3 Registro de uso de IA

Reglas aplicadas: no se ingresan secretos, contraseñas ni datos personales en herramientas de IA, y todo lo que propone una IA se revisa contra el código antes de usarlo. En el repositorio, el código generado con ayuda de IA entra solo por PR revisado.

Tabla 18. Registro de uso de IA.

Fecha	Herramienta	Para qué se usó	Qué se hizo con el resultado	Revisó
03-11-2026	Asistente de IA	Generar código de las funciones del proyecto y ajustarlo	Se revisó por el equipo antes de integrarlo por PR	**Juan Sanhueza**
09-11-2026	Asistente de IA	Revisar el código según los controles de ISO/IEC 27001	Se revisó por el equipo antes de integrarlo por PR	**Juan Sanhueza**

Bibliografía

- Django Software Foundation. (s. f.). Deployment checklist. https://docs.djangoproject.com/en/stable/howto/deployment/checklist/
- GitHub. (s. f.). Planning and tracking with Projects; sub-issues; linking a pull request to an issue; protected branches. GitHub Docs. https://docs.github.com/issues
- ISO/IEC. (2022). ISO/IEC 27001:2022 Information security management systems. Anexo A.
- NIST. (2022). SP 800-218 Secure Software Development Framework (SSDF) Version 1.1. https://csrc.nist.gov/pubs/sp/800/218/final
- OWASP Foundation. (2021). Application Security Verification Standard 4.0.3. https://owasp.org/www-project-application-security-verification-standard/
- OWASP Foundation. (2021). OWASP Top 10:2021. https://owasp.org/Top10/
- Shostack, A. (2014). Threat Modeling: Designing for Security. Wiley.
- Transbank Developers. (s. f.). Webpay Plus. https://www.transbankdevelopers.cl/documentacion/webpay-plus



Anexo A. Backlog completo

Mismo contenido que Backlog_GitHub_Projects_v2.csv y que el Project. Cada fila es un issue del repositorio (columna N.º). Sprint 1 a 3 corresponden a las semanas del 19-10, 26-10 y 02-11-2026; los ítems de los sprints 1 y 2 están en Done y los del sprint 3 entre In progress y Ready.

N.º	Tipo	Título	Sprint	Assignee	Estado, estimación y labels
#5	Épica	**EP1 Organización del equipo**			
#11	Feature	**F1.1 Equipo y roles**			
#21	Historia	P01 Roles del equipo	S1	Díaz	Done; 2 pts
#33	Tarea	Definir roles del equipo	S1	Díaz	Done; 2 h; documentacion
#34	Tarea	Acordar canal y reuniones	S1	Díaz	Done; 1 h; planificacion
#12	Feature	**F1.2 Tablero del proyecto**			
#22	Historia	P02 Armar tablero del proyecto	S3	Ancán	In progress; 3 pts
#35	Tarea	Pedir autorización de GitHub Projects	S3	Díaz	In progress; 1 h; planificacion
#36	Tarea	Crear Project y labels	S3	Ancán	In progress; 2 h; planificacion
#37	Tarea	Cargar backlog e issues	S3	Ancán	In progress; 2 h; planificacion
#38	Tarea	Capturas del tablero	S3	Ancán	Ready; 1 h; evidencia
#6	Épica	**EP2 Requerimientos**			
#13	Feature	**F2.1 Revisión del código**			
#23	Historia	P03 Revisar el código base	S2	Sanhueza	Done; 3 pts
#39	Tarea	Leer documentación del proyecto	S1	Díaz	Done; 3 h; documentacion
#40	Tarea	Levantar con Docker	S1	Ancán	Done; 2 h; entorno
#41	Tarea	Revisión de seguridad del código	S2	Sanhueza	Done; 4 h; seguridad, revision
#14	Feature	**F2.2 Especificación**			
#24	Historia	P04 Definir requerimientos	S2	Yáñez	Done; 5 pts
#42	Tarea	Redactar historias de usuario	S2	Yáñez	Done; 4 h; requerimientos
#43	Tarea	Definir RNF	S2	Yáñez	Done; 2 h; requerimientos
#44	Tarea	Definir requisitos de seguridad	S2	Sanhueza	Done; 4 h; seguridad, requerimientos
#45	Tarea	Ajustar Tarea 2 con feedback	S3	Díaz	Ready; 2 h; requerimientos
#7	Épica	**EP3 Amenazas y proceso**			
#15	Feature	**F3.1 Modelo de amenazas**			
#25	Historia	P05 Modelo de amenazas STRIDE	S3	Sanhueza	In progress; 5 pts
#46	Tarea	Diagrama de arquitectura	S3	Yáñez	In progress; 3 h; seguridad, diseno
#47	Tarea	Amenazas STRIDE	S3	Sanhueza	In progress; 4 h; seguridad, amenazas
#48	Tarea	Matriz amenaza-requerimiento	S3	Sanhueza	In progress; 2 h; seguridad, amenazas
#49	Tarea	Revisar amenazas de PR #1-#3	S3	Sanhueza	In progress; 2 h; seguridad, amenazas
#16	Feature	**F3.2 Proceso DevSecOps**			
#26	Historia	P06 Prácticas por fase	S3	Ancán	In progress; 3 pts
#50	Tarea	Tabla de fases DevSecOps	S3	Ancán	In progress; 3 h; seguridad, devsecops



N.º	Tipo	Título	Sprint	Assignee	Estado, estimación y labels
#8	Épica	**EP4 Repositorio GitHub**			
#17	Feature	**F4.1 Configuración del repo**			
#27	Historia	P07 Ordenar el repositorio	S3	Ancán	In progress; 3 pts
#51	Tarea	Crear repo y ramas	S3	Yáñez	In progress; 1 h; repositorio; Hecho
#52	Tarea	Quitar secretos de .env	S3	Sanhueza	In progress; 2 h; seguridad, secretos
#53	Tarea	Actualizar README	S3	Díaz	In progress; 2 h; documentacion
#54	Tarea	Proteger main y develop	S3	Ancán	Done; 1 h; seguridad, repositorio
#55	Tarea	Acceso de lectura al profe	S3	Ancán	Ready; 1 h; repositorio
#18	Feature	**F4.2 Flujo de trabajo**			
#28	Historia	P08 Flujo de PR e issues	S3	Ancán	In progress; 2 pts
#56	Tarea	Convención de ramas y closes	S3	Ancán	In progress; 1 h; repositorio
#57	Tarea	Plantilla de PR	S3	Sanhueza	Ready; 1 h; seguridad, repositorio
#58	Tarea	Issues para PR #1-#3	S3	Ancán	Ready; 1 h; trazabilidad
#9	Épica	**EP5 Funciones del periodo**			
#19	Feature	**F5.1 Vendedor y catálogo**			
#29	Historia	P09 Modal de vendedor y filtro de precio	S3	Yáñez	In progress; 3 pts
#59	Tarea	Modal vendedor y filtro de precio	S3	Yáñez	In progress; 3 h; frontend, backend; PR #1
#30	Historia	P10 Panel de vendedor y tienda oficial	S3	Yáñez	In progress; 5 pts
#60	Tarea	Panel vendedor y tienda oficial	S3	Yáñez	In progress; 5 h; frontend, backend; PR #2
#61	Tarea	Fix token vencido	S3	Yáñez	In progress; 1 h; seguridad, sesiones; PR #2
#31	Historia	P11 Galería de fotos	S3	Ancán	In progress; 3 pts
#62	Tarea	Galería de hasta 5 fotos	S3	Ancán	In progress; 4 h; frontend, backend; PR #3
#10	Épica	**EP6 Informe Sumativa 1**			
#20	Feature	**F6.1 Informe de planificación**			
#32	Historia	P12 Entregar informe S1	S3	Díaz	In progress; 5 pts
#63	Tarea	Borrador del informe	S3	Sanhueza	Ready; 3 h; documentacion
#64	Tarea	Capturas del Project y del repo	S3	Ancán	Ready; 1 h; evidencia
#65	Tarea	Revisión cruzada del informe	S3	Díaz	Ready; 2 h; documentacion
#66	Tarea	Registro de uso de IA	S3	Sanhueza	Ready; 1 h; documentacion
#67	Tarea	Entregar PDF al profe	S3	Díaz	Ready; 1 h; entrega